

Sentences, Grammar & the Pipeline

Pre-Assessment 2 · 12 questions · 40 points

Name: _____

Date: _____

Score: / 40

This is a **pre-assessment**. It shows what you already know **before** we start this unit, so it is fine if some questions cover things you have not learned yet. Try every question you can. If you have no idea at all, write **“not yet”** and move on.

Work on your own, without a computer. You have about **45 minutes**. When a question asks what a program prints, write exactly what Python would print, one line of output per answer line. If the code would crash, write the name of the error (for example, `TypeError`). Show your thinking; partial credit is given.

In every spaCy question, assume this code has already run: `import spacy` and `nlp = spacy.load("en_core_web_sm")`

1. Sentence splitter

3 points

```
text = "Dr. Lee lives in the U.S.A. now. She likes tea!"
doc = nlp(text)
for sent in doc.sents:
    print(sent.text)
```

(a) Write what the loop prints.

(2 pts)

(b) How many pieces would `text.split(".")` make?

(1 pt)

2. Spans

3 points

```
doc = nlp("My little sister found a golden key.")
span = doc[4:7]
```

Expression	Result
<code>span.text</code>	
<code>span.start</code>	
<code>span.end</code>	

3. Keyword Detective

4 points

Fill in the four blanks so the program prints the 5 most common keywords (as lowercase lemmas), skipping stop words and punctuation.

```
from collections import Counter
doc = nlp(chapter_text)
words = [t._____.lower() for t in doc if not t._____ and not t.is_punct]
counts = _____(words)
print(counts._____(5))
```

4. Parts of speech

4 points

spaCy reads “**The happy puppy quickly chased a red ball.**” Write the `pos_` label for each word. Choose from: **NOUN, VERB, ADJ, ADV.**

Word	happy	puppy	quickly	chased
pos_				

5. Decoding labels

2 points

(a) What kind of word does `spacy.explain("NNS")` describe?

(1 pt)

(b) Which attribute gives detailed labels like `NNS: pos_` or `tag_`?

(1 pt)

6. Noun finder

3 points

```
doc = nlp("The brave girl climbed the tall mountain and saw a golden eagle.")
nouns = [t.text for t in doc if t.pos_ == "NOUN"]
print(nouns)
```

(a) What does it print?

(2 pts)

(b) What would you change so it collects the **adjectives** instead?

(1 pt)

7. Tricky sentences

2 points

spaCy tagged “Fruit flies like a banana.” like this:

SPACY OUTPUT

Fruit NOUN flies VERB like ADP a DET banana NOUN

(a) What part of speech **should** “flies” be here, and what does the sentence mean? (1 pt)

(b) Why did the model make this mistake? (1 pt)

8. Keeping the spaces

4 points

```
doc = nlp("Hi, Sam!")
```

(a) What does `print([t.text_with_ws for t in doc])` show? (2 pts)

(b) What does `print(" ".join([t.text for t in doc]))` show? (1 pt)

(c) Write an expression that rebuilds the original text exactly from the tokens. (1 pt)

9. Who did what?

4 points

spaCy parses “The cat chased the mouse.” Fill in the four blanks. Word bank for `dep_`: **nsubj**, **dobj**, **ROOT**, **det**.

Token	dep_	head.text
The	det	
cat		chased
chased		chased
mouse		chased

10. Sentence Skeletons

2 points

- (a) Write the line of code that draws a dependency diagram of `doc` in a notebook (`displacy` is already imported). (1 pt)

- (b) In “Birds eat seeds.”, list the `children` of the token “eat”. (1 pt)

11. The assembly line

4 points

OUTPUT OF `PRINT(NLP.PIPE_NAMES)`

```
['tok2vec', 'tagger', 'parser', 'attribute_ruler', 'lemmatizer', 'ner']
```

- (a) Which component adds the dependency labels (`dep_`, `head`)? (1 pt)

- (b) Which component finds named entities? (1 pt)

- (c) Why use `nlp.pipe(texts)` instead of calling `nlp(text)` for each text in a loop? (1 pt)

- (d) Write a line that loads `"en_core_web_sm"` with the `ner` component turned off. (1 pt)

12. Most common verbs

5 points

Write a function `top_verbs(texts, n)` that takes a **list** of texts, processes them with `nlp.pipe`, and **returns** the `n` most common **VERB lemmas** (use `Counter`).

EXAMPLE: `TOP_VERBS(["A CAT JUMPED OVER THE FENCE.", "BIRDS SING.", "THE FROG JUMPED."], 1)`
RETURNS

```
[('jump', 2)]
```

END OF ASSESSMENT